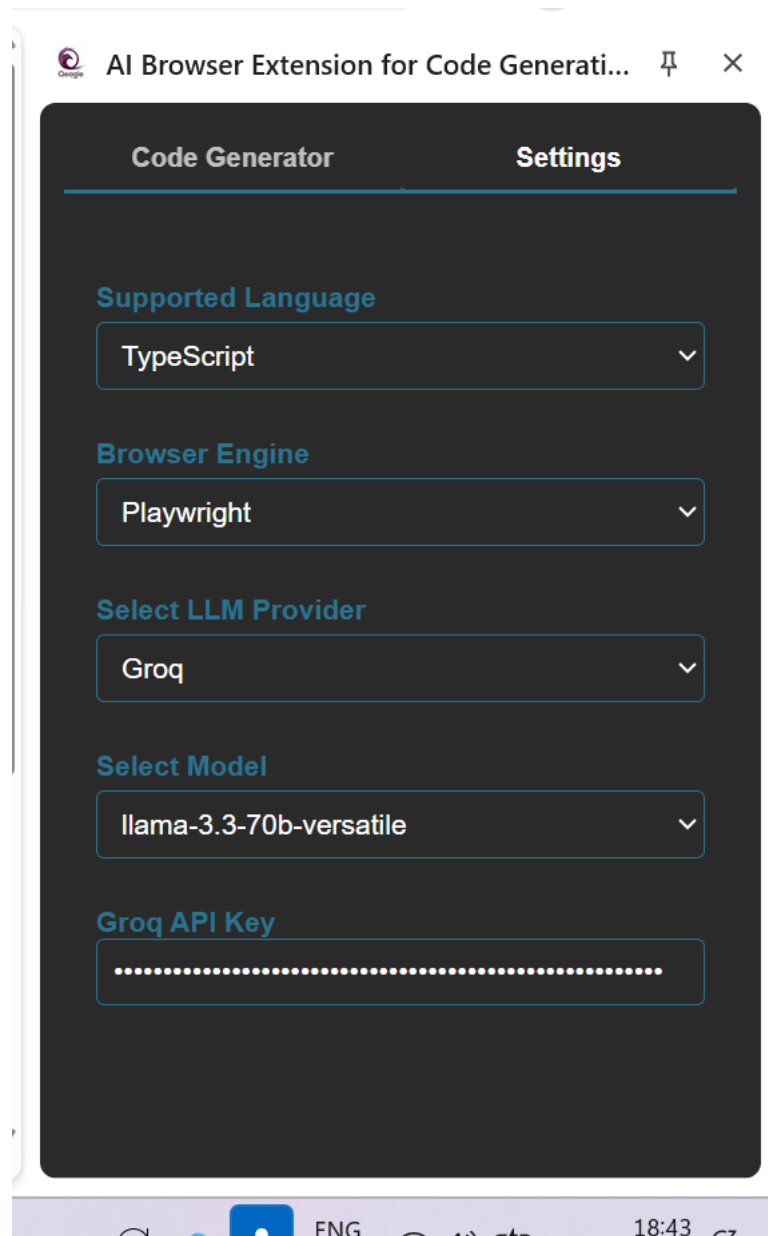
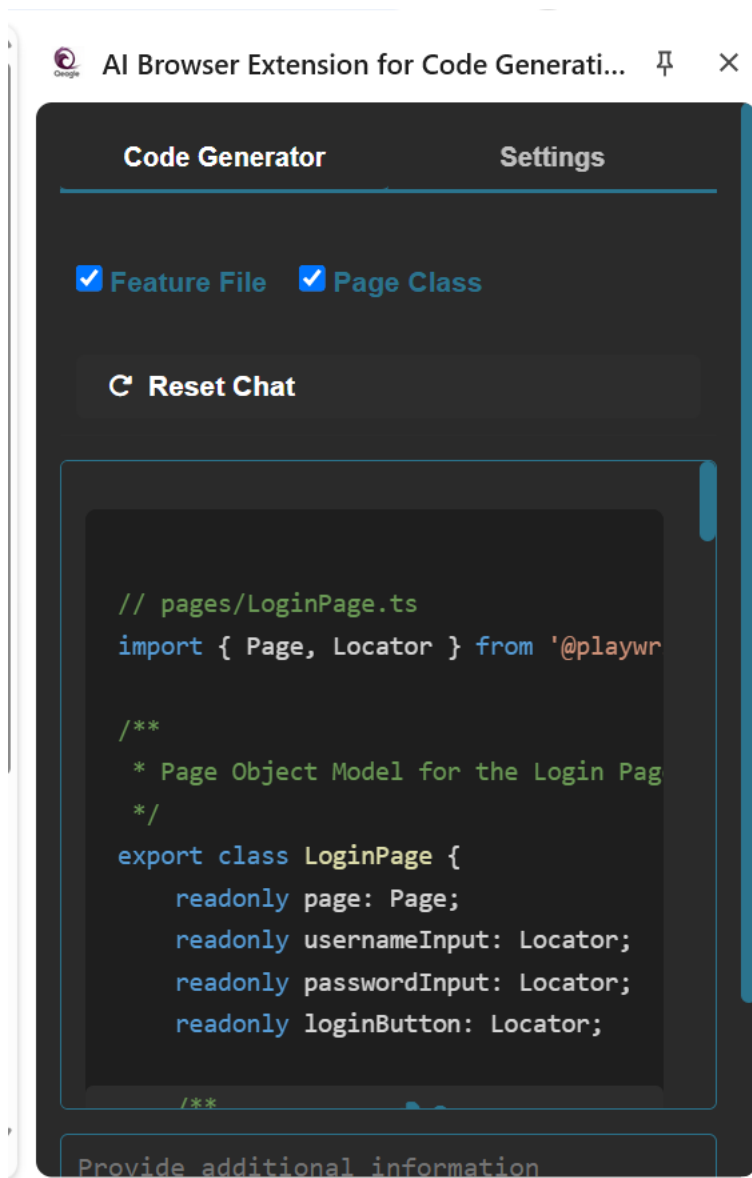


Playwright Integration Using GitHub Copilot





OUTPUT:

```
// pages/LoginPage.ts
import { Page, Locator } from '@playwright/test';

/**
 * Page Object Model for the Login Page.
 */
export class LoginPage {
  readonly page: Page;
  readonly usernameInput: Locator;
  readonly passwordInput: Locator;
  readonly loginButton: Locator;

  /**
   * Initializes the LoginPage object.
```

```

    * @param page The Playwright page object.
    */
    constructor(page: Page) {
        this.page = page;
        this.usernameInput = page.locator('input[id="username"]');
        this.passwordInput = page.locator('input[id="password"]');
        this.loginButton = page.locator('input:has-text("Login")');
    }

    /**
     * Navigates to the login page.
     */
    async goto(): Promise<void> {
        await this.page.goto('https://leftaps.com/opentaps/control/login');
    }

    /**
     * Enters the username.
     * @param username The username to enter.
     */
    async enterUsername(username: string): Promise<void> {
        await this.usernameInput.fill(username);
    }

    /**
     * Enters the password.
     * @param password The password to enter.
     */
    async enterPassword(password: string): Promise<void> {
        await this.passwordInput.fill(password);
    }

    /**
     * Clicks the login button.
     */
    async clickLogin(): Promise<void> {
        await this.loginButton.click();
    }
}

// tests/login.spec.ts
import { test, expect, Page } from '@playwright/test';
import { LoginPage } from '../pages/LoginPage';

/**
 * Test specification for the login functionality.
 */
test.describe('Login to Application', () => {
    let page: Page;
    let loginPage: LoginPage;

    /**
     * Sets up the test environment before each test.
     */

```

```

test.beforeEach(async ({ page: testPage }) => {
  page = testPage;
  loginPage = new LoginPage(page);
  await loginPage.goto();
});

/**
 * Tests a successful login with valid credentials.
 */
test('Successful login with valid credentials', async () => {
  await loginPage.enterUsername('demosalesmanager');
  await loginPage.enterPassword('crmsfa');
  await loginPage.clickLogin();

  await expect(page).toHaveURL('**/dashboard');
});

/**
 * Tests an unsuccessful login with invalid credentials.
 */
test('Unsuccessful login with invalid credentials', async () => {
  await loginPage.enterUsername('invaliduser');
  await loginPage.enterPassword('invalidpassword');
  await loginPage.clickLogin();

  await expect(page).toHaveURL('**/login');
});

/**
 * Tests a login with a valid username and an empty password.
 */
test('Login with valid username and empty password', async () => {
  await loginPage.enterUsername('demosalesmanager');
  await loginPage.enterPassword('');
  await loginPage.clickLogin();

  await expect(page).toHaveURL('**/login');
});

/**
 * Tests a login with an empty username and a valid password.
 */
test('Login with empty username and valid password', async () => {
  await loginPage.enterUsername('');
  await loginPage.enterPassword('crmsfa');
  await loginPage.clickLogin();

  await expect(page).toHaveURL('**/login');
});
});

```